Python vs JavaScript

Detailed Syntax Comparison

■ Python — Indentation-based, dynamic typing, batteries included	■ JavaScript — Brace-based, prototype OOP, async-first, runs in browser & Node

BASICS

Variables & Declarations

■ PYTHON

```
# No keyword needed
x = 10
name = "Alice"
PI = 3.14159
is_valid = True
```

Note: No declaration keyword. Variable type is inferred.

■ JAVASCRIPT

```
// Three declaration keywords
let x = 10; // block-scoped, mutable
const PI = 3.14159; // block-scoped, immutable
var old = "avoid"; // function-scoped (legacy)
```

Note: Prefer const by default, use let when reassignment needed, avoid var.

Data Types

■ PYTHON

```
x = 42 # int
y = 3.14 # float
b = True # bool (capital T/F)
n = None # NoneType
s = "text" # str
type(x) # <class 'int'>
```

Note: Python separates int and float. None is the null equivalent.

■ JAVASCRIPT

```
let x = 42; // number
let y = 3.14; // number (same type!)
let b = true; // boolean (lowercase)
let n = null; // null
let u; // undefined
typeof x; // "number"
```

Note: JS has one number type for both int & float. Has both null and undefined.

Type Coercion & Equality

■ PYTHON

```
# Python does NOT coerce types implicitly
1 == 1 # True
1 == "1" # False (no coercion)
1 is 1 # True (identity check)
```

Note: Python == checks value. is checks identity. No implicit coercion.

■ JAVASCRIPT

```
// JS coerces with ==, use === always
1 == "1" // true (loose, coerces)
1 === "1" // false (strict, no coerce)
1 == true // true (loose coercion!)
```

Note: Always use === in JS. == does type coercion and produces surprising results.

STRINGS

String Basics

■ PYTHON

```
s = "hello"
s = 'hello' # same
multi = """
line one
line two
"""
len(s) # 5
s.upper() # "HELLO"
```

Note: Triple-quoted strings for multiline. len() is a built-in function.

■ JAVASCRIPT

```
let s = "hello";
let s = 'hello'; // same
let multi = `
line one
line two
`;
s.length; // 5 (property, not fn)
s.toUpperCase(); // "HELLO"
```

Note: Template literals (backticks) for multiline. length is a property.

String Interpolation

■ PYTHON

```
name = "Alice"
age = 30
# f-string (modern, preferred)
msg = f"Name: {name}, Age: {age}"
# expressions inside {}
f"Result: {2 ** 10}" # "Result: 1024"
```

Note: f-strings use curly braces {expr}. Added in Python 3.6.

■ JAVASCRIPT

```
const name = "Alice";
const age = 30;
// template literal (modern)
const msg = `Name: ${name}, Age: ${age}`;
// expressions inside ${}
`Result: ${2 ** 10}` // "Result: 1024"
```

Note: Template literals use ${expr}. Both support any expression inside.

String Slicing & Methods

■ PYTHON

```
s = "Hello World"
s[0] # "H"
s[-1] # "d" (negative index!)
s[0:5] # "Hello"
s[::-1] # "dlroW olleH" (reverse)
s.split(" ") # ["Hello", "World"]
s.strip() # removes whitespace
```

Note: Python slicing [start:end:step] is built-in. Negative indexes work everywhere.

■ JAVASCRIPT

```
const s = "Hello World";
s[0] // "H"
s.at(-1) // "d" (.at() for negative)
s.slice(0, 5) // "Hello"
[...s].reverse().join("") // reverse
s.split(" ") // ["Hello", "World"]
s.trim() // removes whitespace
```

Note: No native slice syntax — use .slice() method. Use .at(-1) for negative indexing.

FUNCTIONS

Defining Functions

■ PYTHON

```
def greet(name):
return f"Hello, {name}!"

# with type hints (optional)
def add(a: int, b: int) -> int:
return a + b
```

Note: Indentation defines the function body. def keyword. Type hints are optional.

■ JAVASCRIPT

```
// function declaration
function greet(name) {
return `Hello, ${name}!`;
}
// function expression
const add = function(a, b) {
return a + b;
}
```

Note: Curly braces define the body. Declarations are hoisted; expressions are not.

Arrow / Lambda Functions

■ PYTHON

```python
# lambda: single expression only
square = lambda x: x ** 2
add = lambda a, b: a + b

# used inline
nums = [3,1,2]
sorted(nums, key=lambda x: -x)
```

Note: lambda is limited to one expression. For multi-line, use def.

■ JAVASCRIPT

```javascript
// arrow functions: full-featured
const square = x => x ** 2;
const add = (a, b) => a + b;

// multi-line arrow function
const greet = name => {
return `Hello ${name}`;
};
```

Note: Arrow functions have implicit return for single expressions. They don't bind their own 'this'.

Default & Keyword Arguments

■ PYTHON

```python
def greet(name, greeting="Hello"):
return f"{greeting}, {name}!"

greet("Alice") # uses default
greet("Bob", "Hi") # positional
greet(greeting="Hey", name="Eve") # kwargs
```

Note: Python supports named (keyword) arguments anywhere in the call.

■ JAVASCRIPT

```javascript
function greet(name, greeting = "Hello") {
return `${greeting}, ${name}!`;
}

greet("Alice"); // uses default
greet("Bob", "Hi"); // positional
// No named/keyword args in JS
```

Note: JS has default parameters but no named argument support. Simulate with destructured objects.

*args / rest & **kwargs / spread

■ PYTHON

```python
def total(*nums): # *args tuple
return sum(nums)

def info(**details): # **kwargs dict
for k, v in details.items():
print(f"{k}={v}")

info(name="Ali", age=25)
```

*Note: *args collects into a tuple. **kwargs collects into a dict.*

■ JAVASCRIPT

```javascript
function total(...nums) { // rest params -> array
return nums.reduce((a,b) => a+b, 0);
}

// No **kwargs - simulate with object
function info({ name, age }) {
console.log(`${name}, ${age}`);
}
info({ name: "Ali", age: 25 });
```

*Note: ...rest collects into an array. No equivalent to **kwargs — use object destructuring.*

CONTROL FLOW

if / elif / else

■ PYTHON

```python
if x > 10:
print("big")
elif x > 5:
print("medium")
else:
print("small")
```

Note: elif (not 'else if'). Colon + indentation instead of braces.

■ JAVASCRIPT

```javascript
if (x > 10) {
console.log("big");
} else if (x > 5) {
console.log("medium");
} else {
console.log("small");
}
```

Note: else if (two words). Condition must be in parentheses. Curly braces define blocks.

Ternary / Conditional Expression

■ PYTHON

```python
# value_if_true if condition else value_if_false
grade = "pass" if score >= 50 else "fail"
label = "even" if n % 2 == 0 else "odd"
```

Note: Python reads more like English: result if condition else fallback.

■ JAVASCRIPT

```javascript
// condition ? value_if_true : value_if_false
const grade = score >= 50 ? "pass" : "fail";
const label = n % 2 === 0 ? "even" : "odd";
```

Note: Classic C-style ternary. Condition first, then ? trueValue : falseValue.

---

match / switch

■ PYTHON

```python
# match (Python 3.10+)
match command:
case "quit":
quit()
case "help":
show_help()
case _: # default
print("unknown")
```

Note: match-case added in Python 3.10. The _ is the wildcard/default. No break needed.

■ JAVASCRIPT

```javascript
switch (command) {
case "quit":
quit();
break; // REQUIRED to stop fallthrough
case "help":
showHelp();
break;
default:
console.log("unknown");
}
```

Note: switch requires explicit break or execution falls through to next case.

LOOPS

for Loop

■ PYTHON

```python
# iterate over sequence directly
for item in [1, 2, 3]:
print(item)

# with index
for i, val in enumerate(["a", "b"]):
print(i, val)

# range
for i in range(5):
print(i)
```

Note: Python for loops iterate over anything iterable. enumerate() for index+value.

■ JAVASCRIPT

```javascript
// classic C-style
for (let i = 0; i < 3; i++) {
console.log(i);
}

// for...of (iterate values)
for (const item of [1, 2, 3]) {
console.log(item);
}

// entries() for index
for (const [i, v] of arr.entries()) {
console.log(i, v);
}
```

Note: for...of is the modern Python-like loop. for...in iterates object keys.

---

while Loop

■ PYTHON

```python
count = 0
while count < 5:
print(count)
count += 1

# while with else (unique to Python!)
while count < 5:
count += 1
else:
print("done") # runs if no break
```

Note: Python while supports optional else block. No ++ operator — use count += 1.

■ JAVASCRIPT

```javascript
let count = 0;
while (count < 5) {
console.log(count);
count++;
}

// do...while (runs at least once)
do {
count++;
} while (count < 5);
```

Note: JS has do...while (body runs before condition check). Also has ++ increment operator.

List / Array Comprehensions

■ PYTHON

```python
nums = [1,2,3,4,5]

# list comprehension
squares = [x**2 for x in nums]

# with filter
evens = [x for x in nums if x % 2 == 0]

# dict comprehension
sq_map = {x: x**2 for x in nums}
```

Note: List comprehensions are concise and idiomatic Python. Also: set & dict comprehensions.

■ JAVASCRIPT

```javascript
const nums = [1,2,3,4,5];

// .map() equivalent
const squares = nums.map(x => x**2);

// .filter() equivalent
const evens = nums.filter(x => x%2===0);

// Object.fromEntries()
const sqMap = Object.fromEntries(
nums.map(x => [x, x**2])
);
```

Note: JS uses .map(), .filter(), .reduce() for functional transforms. No comprehension syntax.

COLLECTIONS

Lists vs Arrays

■ PYTHON

```python
lst = [1, "two", 3.0] # mixed types OK
lst.append(4) # add to end
lst.insert(0, "zero") # insert at index
lst.pop() # remove last
lst.pop(0) # remove at index
lst.remove("two") # remove by value
lst.sort()
len(lst) # length
```

Note: Python uses append/pop/insert/remove. len() is a built-in function.

■ JAVASCRIPT

```javascript
const arr = [1, "two", 3.0]; // mixed OK
arr.push(4); // add to end
arr.unshift("zero"); // insert at start
arr.pop(); // remove last
arr.shift(); // remove first
arr.splice(i, 1); // remove at index
arr.sort();
arr.length; // length (property)
```

Note: JS arrays use push/pop (end) and unshift/shift (start). splice for arbitrary index.

Dictionaries vs Objects

■ PYTHON

```python
person = {"name": "Alice", "age": 30}
person["email"] = "a@b.com" # add key
del person["age"] # delete key
"name" in person # True
person.get("x", "default") # safe access
person.keys()
person.values()
person.items() # key-value pairs
```

Note: dict keys can be any hashable type. .get() is the safe way to access with a default.

■ JAVASCRIPT

```javascript
const person = { name: "Alice", age: 30 };
person.email = "a@b.com"; // add key
delete person.age; // delete key
"name" in person; // true
person.x ?? "default"; // safe access
Object.keys(person);
Object.values(person);
Object.entries(person);
```

Note: Object keys are strings/symbols. Use ?? for nullish coalescing.

Sets

■ PYTHON

```python
s = {1, 2, 3}
s.add(4)
s.discard(2)
3 in s # True

a = {1,2,3}; b = {2,3,4}
a & b # {2,3} intersection
a | b # {1,2,3,4} union
a - b # {1} difference
```

Note: Python sets have built-in operators for union, intersection, difference.

■ JAVASCRIPT

```javascript
const s = new Set([1, 2, 3]);
s.add(4);
s.delete(2);
s.has(3); // true

// Set operations (ES2025+)
a.intersection(b);
a.union(b);
a.difference(b);
// Or use spread + filter for older envs
```

Note: Set operations like .union()/.intersection() are new in ES2025.

OOP

Classes & Constructors

■ PYTHON

```python
class Animal:
def __init__(self, name, sound):
self.name = name
self.sound = sound

def speak(self):
return f"{self.name} says {self.sound}"

dog = Animal("Rex", "Woof")
dog.speak()
```

Note: __init__ is the constructor. self is required as the first parameter of every instance method.

■ JAVASCRIPT

```javascript
class Animal {
constructor(name, sound) {
this.name = name;
this.sound = sound;
}
speak() {
return `${this.name} says ${this.sound}`;
}
}

const dog = new Animal("Rex", "Woof");
dog.speak();
```

Note: constructor() is the constructor. this refers to the instance. new keyword required.

Inheritance

■ PYTHON

```
class Dog(Animal): # extends Animal
def __init__(self, name, breed):
super().__init__(name, "Woof")
self.breed = breed

def info(self):
return f"{self.name} ({self.breed})"

isinstance(dog, Animal) # True
```

Note: Pass parent class in parentheses: class Dog(Animal). super().__init__() calls parent.

■ JAVASCRIPT

```
class Dog extends Animal {
constructor(name, breed) {
super(name, "Woof"); // call parent
this.breed = breed;
}
info() {
return `${this.name} (${this.breed})`;
}
}
dog instanceof Animal; // true
```

Note: extends keyword. super() must be called before using this in the child constructor.

Private & Static Members

■ PYTHON

```
class Counter:
_count = 0 # convention: private
__secret = 42 # name-mangled

@classmethod
def increment(cls):
cls._count += 1

@staticmethod
def reset():
Counter._count = 0
```

Note: Python has no true private fields. _ is convention, __ triggers name mangling only.

■ JAVASCRIPT

```
class Counter {
#count = 0; // truly private (#)
static total = 0; // static field

increment() {
this.#count++;
Counter.total++;
}

static reset() {
Counter.total = 0;
}
}
```

Note: # prefix creates truly private fields (ES2022). static keyword for static properties/methods.

ASYNC

Async / Await

■ PYTHON

```
import asyncio

async def fetch_data(url):
await asyncio.sleep(1) # non-blocking
return "data"

async def main():
result = await fetch_data("url")
print(result)

asyncio.run(main())
```

Note: Python async requires asyncio event loop. asyncio.run() starts it.

■ JAVASCRIPT

```
async function fetchData(url) {
await new Promise(r => setTimeout(r, 1000));
return "data";
}

async function main() {
const result = await fetchData("url");
console.log(result);
}

main(); // top-level await in modules
```

Note: JS is single-threaded with a built-in event loop. async/await wraps Promises.

Parallel Execution

■ PYTHON

```python
import asyncio

async def main():
# run tasks concurrently
a, b = await asyncio.gather(
task_one(),
task_two()
)
```

Note: asyncio.gather() runs coroutines concurrently and returns all results.

■ JAVASCRIPT

```javascript
// Promise.all() for parallel
const [a, b] = await Promise.all([
taskOne(),
taskTwo()
]);

// Promise.allSettled() - never throws
const results = await Promise.allSettled([...]);
```

Note: Promise.all() fails fast on any rejection. Promise.allSettled() waits for all.

ERROR HANDLING

try / except / catch

■ PYTHON

```python
try:
result = 10 / 0
except ZeroDivisionError as e:
print(f"Error: {e}")
except (TypeError, ValueError) as e:
print("Type/Value error")
else:
print("no error") # runs if no except
finally:
print("always runs")
```

Note: Python allows multiple except clauses and an else block (runs only if no exception).

■ JAVASCRIPT

```javascript
try {
const result = 10 / 0;
} catch (e) {
// check manually
if (e instanceof TypeError) { ... }
console.error(e.message);
} finally {
console.log("always runs");
}
// No else block in JS try/catch
```

Note: JS has one catch block for all errors. Use instanceof to differentiate error types.

Raising / Throwing Errors

■ PYTHON

```python
raise ValueError("bad input")
raise TypeError("wrong type")

# Custom exception
class AppError(Exception):
def __init__(self, msg, code):
super().__init__(msg)
self.code = code

raise AppError("oops", 404)
```

Note: raise keyword (not throw). Python has many built-in exception types.

■ JAVASCRIPT

```javascript
throw new Error("bad input");
throw new TypeError("wrong type");

// Custom error class
class AppError extends Error {
constructor(msg, code) {
super(msg);
this.code = code;
}
}

throw new AppError("oops", 404);
```

Note: throw keyword. new keyword required. All errors should extend Error for stack traces.

MODULES

Importing

■ PYTHON

```python
# import whole module
import math
math.sqrt(16)

# import specific names
from math import sqrt, pi

# alias
import numpy as np

# relative import
from .utils import helper
```

Note: Python imports are path-based. from X import Y brings names into scope directly.

■ JAVASCRIPT

```javascript
// named imports
import { sqrt, PI } from "./math.js";

// default import
import React from "react";

// namespace import
import * as np from "./math.js";

// dynamic import
const mod = await import("./mod.js");
```

Note: ES Modules use import/export. Dynamic import() is async and returns a Promise.

Exporting

■ PYTHON

```python
# utils.py - just define, no export keyword
def helper():
return "help"

MY_CONST = 42

# Control public API with __all__
__all__ = ["helper", "MY_CONST"]
```

Note: In Python, everything in a module is importable by default. __all__ restricts wildcard imports.

■ JAVASCRIPT

```javascript
// Named exports
export function helper() { return "help"; }
export const MY_CONST = 42;

// Default export (one per file)
export default function main() { ... }

// Re-export
export { helper } from "./utils.js";
```

Note: Explicit export keyword required. One default export per file, unlimited named exports.